

BST Identification Database

Data Structures and Algorithms Project

Project

Implementation and analysis of a Binary Search Tree (BST) for a government identification database, including inorder traversal, a BST vs linear search comparison, and an analysis of the effect of key length and insertion order.

Question

A government database stores the identification numbers below. Implement a BST using lexicographic string comparison, perform inorder traversal, compare BST search with linear search, and analyse the effect of key length and insertion order. Suggest a suitable approach for a growing database.

Input

A102, A25, A7, B100, B12, A120, B3, A45

Search Keys

A102, A120, A45, B12, B3, A7

Folder Structure

```
01_Question/  
02_Source_Code/  
03_Input_Data/  
04_Output/  
05_Trace_Tables/  
06_Complexity_Analysis/  
07_Comparison/  
08_Final_Conclusion/
```

Each folder contains the corresponding item required for the GitHub submission.

Final Inorder Traversal

A102 A120 A25 A45 A7 B100 B12 B3

Resulting BST height = 5 edges (unbalanced for the given insertion order).

1. Source Code

File: bst_identification.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ID 20
#define N 8

struct Node {
    char id[MAX_ID];
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(const char id[]) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    strcpy(newNode->id, id);
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *root, const char id[]) {
    if (root == NULL)
        return createNode(id);

    if (strcmp(id, root->id) < 0)
        root->left = insert(root->left, id);
    else if (strcmp(id, root->id) > 0)
        root->right = insert(root->right, id);

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%s ", root->id);
        inorder(root->right);
    }
}

int bstSearch(struct Node *root, const char key[]) {
    int comparisons = 0;

    while (root != NULL) {
        comparisons++;

        if (strcmp(key, root->id) == 0)
            return comparisons;

        if (strcmp(key, root->id) < 0)
            root = root->left;
        else
            root = root->right;
    }

    return comparisons;
}

int linearSearch(const char ids[][MAX_ID], int n, const char key[]) {
    int comparisons = 0;

    for (int i = 0; i < n; i++) {
        comparisons++;
        if (strcmp(ids[i], key) == 0)
            return comparisons;
    }
}
```

```

    }

    return comparisons;
}

int height(struct Node *root) {
    if (root == NULL)
        return -1;

    int left = height(root->left);
    int right = height(root->right);

    return (left > right ? left : right) + 1;
}

void freeTree(struct Node *root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main(void) {
    const char ids[N][MAX_ID] = {
        "A102", "A25", "A7", "B100",
        "B12", "A120", "B3", "A45"
    };

    const char searchKeys[][MAX_ID] = {
        "A102", "A120", "A45", "B12", "B3", "A7"
    };

    int searchCount = sizeof(searchKeys) / sizeof(searchKeys[0]);
    struct Node *root = NULL;

    printf("BST IDENTIFICATION DATABASE\n");
    printf("=====\n\n");

    printf("Input identification numbers:\n");
    for (int i = 0; i < N; i++)
        printf("%s ", ids[i]);

    for (int i = 0; i < N; i++)
        root = insert(root, ids[i]);

    printf("\n\nInorder traversal:\n");
    inorder(root);

    printf("\n\nBST height (edges) = %d\n", height(root));

    printf("\nSearch comparison table:\n");
    printf("%-8s %-14s %-16s\n", "ID", "BST", "Linear");
    printf("-----\n");

    for (int i = 0; i < searchCount; i++) {
        int bst = bstSearch(root, searchKeys[i]);
        int linear = linearSearch(ids, N, searchKeys[i]);

        printf("%-8s %-14d %-16d\n",
            searchKeys[i], bst, linear);
    }

    freeTree(root);
    return 0;
}

```

2. Input Data

Identification numbers are inserted in the order shown (one per line in `input.txt`).

Insertion Order	Identification Number
1	A102
2	A25
3	A7
4	B100
5	B12
6	A120
7	B3
8	A45

Raw input file

```
A102
A25
A7
B100
B12
A120
B3
A45
```

3. Output

Program output (compiled with gcc and executed; matches the supplied output exactly):

```
BST IDENTIFICATION DATABASE
=====

Input identification numbers:
A102 A25 A7 B100 B12 A120 B3 A45

Inorder traversal:
A102 A120 A25 A45 A7 B100 B12 B3

BST height (edges) = 5

Search comparison table:
ID      BST      Linear
-----
A102    1         1
A120    3         6
A45     4         8
B12     5         5
B3      6         7
A7      3         3
```

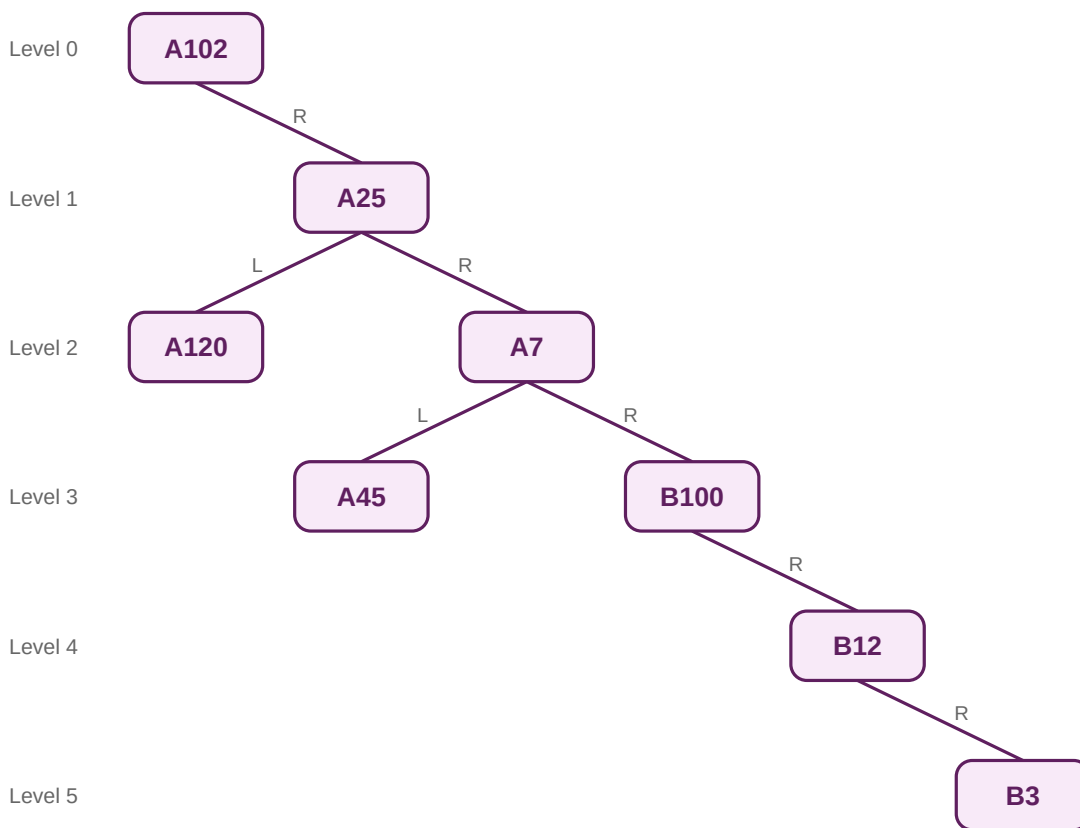
4. Trace Tables

4.1 BST Insertion Trace

Step	Inserted ID	Comparisons / Path	Position
1	A102	-	Root
2	A25	A25 > A102	Right of A102
3	A7	A7 > A102, A7 > A25	Right of A25
4	B100	B100 > A102, B100 > A25, B100 > A7	Right of A7
5	B12	B12 > A102, B12 > A25, B12 > A7, B12 > B100	Right of B100
6	A120	A120 > A102, A120 < A25	Left of A25
7	B3	B3 > A102, B3 > A25, B3 > A7, B3 > B100, B3 > B12	Right of B12
8	A45	A45 > A102, A45 > A25, A45 < A7	Left of A7

Comparisons use lexicographic (character-by-character) string order, e.g. "A120" > "A102" because '2' > '0' at the fourth character, and "A120" < "A25" because '1' < '2' at the second character.

4.2 Resulting BST



Height = 5 edges (6 levels). Every node except A25 and A7 has only one child, so the tree is heavily skewed to the right.

4.3 BST Search Trace

Key	BST Path (FOUND at last node)	BST Comps	Linear Comps
A102	A102	1	1
A120	A102 -> A25 -> A120	3	6
A45	A102 -> A25 -> A7 -> A45	4	8
B12	A102 -> A25 -> A7 -> B100 -> B12	5	5
B3	A102 -> A25 -> A7 -> B100 -> B12 -> B3	6	7
A7	A102 -> A25 -> A7	3	3

5. Complexity Analysis

Let n be the number of identification numbers and h the height of the BST.

Operation	Best	Average	Worst
BST search	$O(1)$ - key at root	$O(\log n)$ - balanced	$O(n)$ - skewed
BST insertion	$O(1)$ - empty tree	$O(\log n)$ - balanced	$O(n)$ - skewed
Inorder traversal	$O(n)$	$O(n)$	$O(n)$
Linear search	$O(1)$	$O(n)$	$O(n)$
BST storage	$O(n)$	$O(n)$	$O(n)$

Additional notes

- **Recursion space:** inorder traversal uses $O(h)$ auxiliary stack space, where h is the tree height.
- **Effect of key length:** BST height depends mainly on the number of keys and their insertion order. Longer identification strings increase the cost of each string comparison, because characters are compared until a difference is found.
- **Effect of insertion order:** an ordinary BST is strongly affected by insertion order. A suitable order gives a more balanced tree; an unfortunate order gives a skewed tree with height close to $n-1$.
- **Theoretical comparison:** balanced BST search $O(\log n)$; skewed BST search $O(n)$; linear search $O(n)$.
- **Growing database:** a self-balancing BST such as an AVL tree or Red-Black tree maintains logarithmic-height search performance.

6. Performance Comparison

Search ID	BST Comparisons	Linear Search Comparisons	Result
A102	1	1	Equal
A120	3	6	BST faster
A45	4	8	BST faster
B12	5	5	Equal
B3	6	7	BST faster
A7	3	3	Equal

Observation

- BST is faster for A120, A45 and B3 in this experiment.
- BST and linear search use the same number of comparisons for A102, B12 and A7.
- The ordinary BST is unbalanced for the given insertion order, so its performance is not always close to $O(\log n)$.

7. Final Conclusion

The given identification numbers were inserted into a Binary Search Tree using lexicographic string comparison. The inorder traversal produced the identifiers in sorted (lexicographic) order:

A102 A120 A25 A45 A7 B100 B12 B3

Because the comparison is character by character, this is dictionary order rather than numeric order (for example, A25 comes after A120).

The given insertion order produced an unbalanced BST with height 5 edges. The search experiment showed that BST search used fewer comparisons for several of the selected keys, although it was equal to linear search for some keys.

Key length mainly affects the cost of individual string comparisons, whereas insertion order has a direct effect on BST height. A balanced BST provides $O(\log n)$ search in the usual balanced case, while an ordinary skewed BST can degrade to $O(n)$, which is the same asymptotic worst case as linear search.

Therefore, as the identification database grows, a self-balancing BST such as an AVL tree or Red-Black tree is a suitable approach for maintaining efficient searches. For very large persistent databases, B-tree / B+ tree based indexing is also commonly suitable.